

SD3589CV100 SMR API
开发参考

文档版本　　1.0.0

发布日期　　2025-06-12

目录

前　言 iv
1. 概述 1
2. 重要概念 2
3. 功能描述 3
   3.1. 功能概述 3
   3.2. 功能框图 3
   3.3. 管理内存地址空间配置 3
   3.4. DEMO板上DDR内存管理示例 4
4. API 参考 5
   4.1. ot_smr_init 5
   4.2. ot_smr_deinit 5
   4.3. ot_smr_alloc 6
   4.4. ot_smr_free 7
   4.5. ot_smr_mmap 8
   4.6. ot_smr_munmap 9
   4.7. ot_smr_virt2phy 10
   4.8. ot_smr_is_cached 11
   4.9. ot_smr_flush_cache 12
5. 数据类型 14
   5.1. ot_smr_alloc_attr 14
6. 错误码 1

版权所有 © 上海云集互联有限公司

前 言

概述

本文档主要介绍 SD3589CV100芯片上SMR API的使用。

版本说明

与本文档相对应的产品版本如下。

产品名称	产品版本
SD3589C	V100
SD3589D	V100

用户

本文档主要适用于以下工程师：

- ✧ 技术支持工程师
- ✧ 软件工程师
- ✧ 系统工程师

修订记录

修订记录列出了各文档版本间发生的主要更改。最新版本的文档包含以前所有文档版本的更新内容。

版本	修订日期	修订说明
v1.0.0	2025.06.12	首个SMR API接口文档

版权所有 © 上海云集互联有限公司

1. 概述

SMR（Special Memory Region）模块实现的功能：管理一段连续物理内存资源，支撑媒体和相关业务对连续物理内存的诉求

版权所有 © 上海云集互联有限公司

2. 重要概念

- 物理地址

物理地址标记一块实际物理内存空间的地址信息

- 虚拟地址

虚拟地址为进程空间内程序可直接操作的逻辑地址，基于指定的有效物理地址映射生成

版权所有 © 上海云集互联有限公司

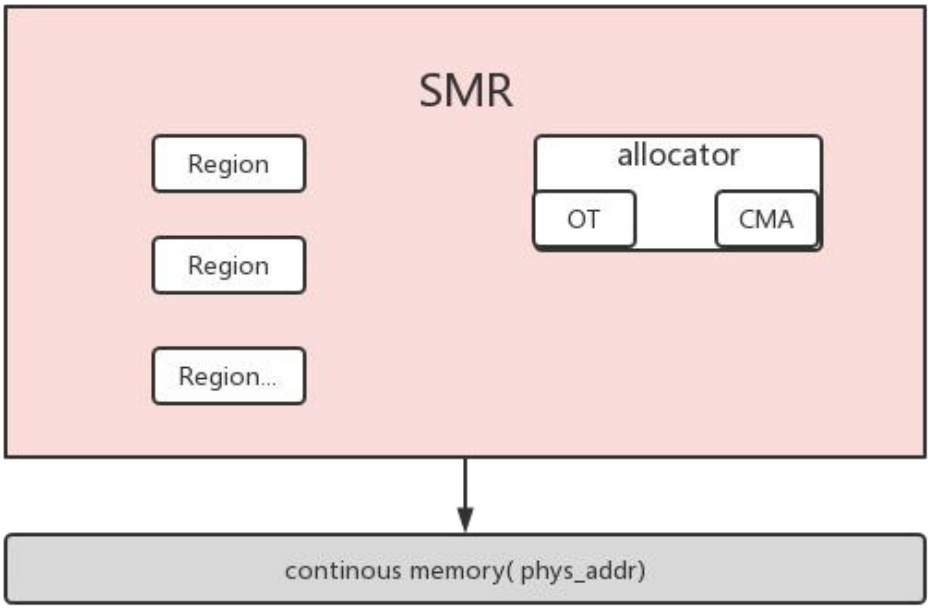
3. 功能描述

3.1. 功能概述

SMR管理一段连续的DDR内存，面向业务层和媒体处理模块提供连续物理内存空间的诉求支撑。主要具备以下功能：

（1）连续物理内存申请
（2）连续物理内存释放
（3）内存映射
（4）内存去映射
（5）内存属性查询
（6）内存缓存

3.2. 功能框图



如图，SMR下基于region的形式进行管理，其中每个region代表一段连续物理内存空间。基于不同场景诉求，客户可以进行定制化配置。即供媒体使用的DDR内存可以放在一个region，也可以放在多个region。当前配置下需要指定一个名为anonymous的匿名region，匿名region作为默认区域，在未指定region进行内存申请时，SMR会从默认region进行申请。

3.3. 管理内存地址空间配置

所有的DDR内存中，一部分由操作系统管理，成为OS内存。一部分由SMR模块管理，供媒体模块单独使用，成为SMR内存。SMR内存由内核模块管理（main/out/ko目录下的ot_smr.ko），加载ot_smr.ko模块时，通过模块参数指定其region属性（region name & allocator & 起始地址 &大小）。需要注意的时SMR的内存地址范围不可以与OS内存重叠。

版权所有 © 上海云集互联有限公司

3.4. DEMO板上DDR内存管理示例

以容量为1G Bytes的DDR内存为例，以下为根据本文档和SDK默认配置得到的内存管理示意图。

DDR:

```
|---------------|--------------------| 0x40000000 # Memory managed by FDT and ATF.
| 192KB | FDT/ATF |
|---------------|--------------------| 0x40030000 # Memory managed by Linux OS.
| 1856KB+128MB | Linux OS |
|---------------|--------------------| 0x48200000 # Memory managed by SMR block anonymous.
| 894MB | SMR |
|------------- |--------------------| 0x80000000 # End of memory managed by SMR.
```

1. 用户在配置启动参数时需要设置OS的管理内存为132928KB，“ setenv bootargs 'mem=132928K...”。
2. 在load_SD3589Cv100脚本中，默认smr_start配置为0x48200000， smr_size配置为894M，如果修改了默认的内存布局，要同步修改脚本中的smr_start和smr_size。
3. 如果有特殊应用，可以自行修改load_SD3589Cv100脚本，进行smr区域划分，如“insmod ot_smr.ko anony=1 smr_allocator=ot_smr=anonymous,0,$smr_start,$smr_size$1 mem_process_isolation=0”（mem_process_isolation当前版本只支持0，anony必须为1）

版权所有 © 上海云集互联有限公司

4. API 参考

SMR模块提供连续物理内存的申请、释放、映射、去映射，缓存和查询等功能。
该功能模块提供以下API：

- ot_smr_init：smr初始化。
- ot_smr_deinit：smr去初始化。
- ot_smr_alloc：申请物理内存。
- ot_smr_free：释放物理内存。
- ot_smr_mmap：映射一段物理内存。
- ot_smr_munmap：去映射一段物理内存。
- ot_smr_virt2phy：基于虚拟地址查找对应的物理地址。
- ot_smr_is_cached：查询对应内存地址区间是否缓存。
- ot_smr_flush_cache：清空对应内存地址空间的缓存。

4.1. ot_smr_init

【描述】
smr初始化。
【语法】

```
ot_s32 ot_smr_init(ot_void);
```

【参数】

【返回值】

返回值	描述
0	成功。
非0	失败，其值为错误码。

【需求】

- 头文件：ot_smr_common.h、ot_smr.h
- 库文件：libot_smr.a

【注意】

- 在进行smr内存操作前必须先进行初始化(每个进程都需要)
- 支持多次初始化，首次初始化完成smr模块初始化，后续直接返回OK

【举例】无。
【相关主题】

4.2. ot_smr_deinit

【描述】
smr去初始化。
【语法】

```
ot_s32 ot_smr_deinit(ot_void);
```


版权所有 © 上海云集互联有限公司

【参数】

【返回值】

返回值	描述
0	成功。
非0	失败，其值为错误码。

【需求】

- 头文件：ot_smr_common.h、ot_smr.h
- 库文件：libot_smr.a

【注意】

- 业务进程退出前需要进行一次去初始化（如果有做ot_smr_init）。
- 支持多次去初始化，首次清理资源，后续调用直接返回OK。

【举例】无。

【相关主题】

ot_smr_init

4.3. ot_smr_alloc

【描述】

申请连续物理内存。

【语法】

```
ot_s32 ot_smr_alloc(const ot_smr_alloc_attr* attr, ot_phys_addr* phy_addr, ot_void**
virt_addr);
```

【参数】

版权所有 © 上海云集互联有限公司

参数名称	描述	输入/输出
attr	内存申请请求信息，包括指定的region name、len、align和cached配置	输入
phy_addr	内存物理地址	输出
virt_addr	虚拟地址（进程空间内）	输出

【返回值】

返回值	描述
0	成功。
非0	失败，其值为错误码。

【需求】

- 头文件：ot_smr_common.h、ot_smr.h
- 库文件：libot_smr.a

【注意】

- 当前进程内需要先完成至少一次ot_smr_init初始化操作。
- region name必须为smr加载时候配置的对应region name，如果没有强要求，直接填空或者anony，smr会从默认区域分配物理内存
- cache属性为设置是否对申请的内存进行缓存，缓存有助于提升访问效率
- len的单位是字节
- 申请的内存在使用完之后必须在当前进程内通过ot_smr_free进行释放，否则会造成内存泄露

【举例】无。

【相关主题】

ot_smr_free

4.4. ot_smr_free

【描述】

释放申请的物理内存。

【语法】

```
ot_s32 ot_smr_free(ot_phys_addr phy_addr);
```

【参数】

参数名称	描述	输入/输出

版权所有 © 上海云集互联有限公司

phy_addr	内存物理地址	输入

【返回值】

返回值	描述
0	成功。
非0	失败，其值为错误码。

【需求】

- 头文件：ot_smr_common.h、ot_smr.h
- 库文件：libot_smr.a

【注意】

- 物理地址可以是ot_smr_alloc申请的内存块中的任意地址。
- 不支持跨进程释放，即A进程申请的内存必须由A进程进行释放。
- 不能释放正在进行flush操作的内存，否则会引发不可预知的异常。

【举例】无。

【相关主题】

ot_smr_alloc

4.5. ot_smr_mmap

【描述】

映射一段物理内存空间。

【语法】

```
ot_s32 ot_smr_mmap(ot_phys_addr phy_addr, ot_u32 size, ot_bool cached, ot_void** virt_addr)
```

【参数】

参数名称	描述	输入/输出
phy_addr	物理内存起始地址，由ot_smr_alloc申请返回的物理地址	输入
size	要映射的物理内存的大小，单位字节	输入
cached	设置是否缓存（缓存提升内存访问效率）	输入
virt_addr	映射之后的虚拟地址	输出

【返回值】

版权所有 © 上海云集互联有限公司

返回值	描述
0	成功。
非0	失败，其值为错误码。

【需求】

- 头文件：ot_smr_common.h、ot_smr.h
- 库文件：libot_smr.a

【注意】

- 物理地址可以是ot_smr_alloc申请的内存块中的任意地址，但是size不能超过申请的内存块地址范围。
- 用户态映射属性需要与内核态映射属性保持一致。
- 不支持跨进程映射，即A进程申请的内存不可以在B进程进行映射，只支持在进程内申请释放

【举例】无。

【相关主题】

ot_smr_alloc

4.6. ot_smr_munmap

【描述】

去映射一段内存。

版权所有 © 上海云集互联有限公司

【语法】

```
ot_s32 ot_smr_munmap(ot_void* virt_addr, ot_u32 size);
```

【参数】

参数名称	描述	输入/输出
virt_addr	虚拟地址，由ot_smr_mmap 映射生成的虚拟地址，虚拟地址可为内块空间内的任意地址	输入
size	去映射内存空间大小，虚拟地址加上size不能超过已申请的内存空间的大小。	输入

【返回值】

返回值	描述
0	成功。
非0	失败，其值为错误码。

【需求】

- 头文件：ot_smr_common.h、ot_smr.h
- 库文件：libot_smr.a

【注意】

等同于ot_smr_mmap。

【举例】无。
【相关主题】

ot_smr_mmap

4.7. ot_smr_virt2phy

【描述】
基于虚拟地址查找对应的物理地址。
【语法】

```
ot_s32 ot_smr_virt2phy(ot_void* virt_addr, ot_phys_addr* phy_addr);
```

参数名称	描述	输入/输出
virt_addr	虚拟地址，由ot_smr_mmap 映射生成的虚拟地址，虚拟地址可为内块空间内的任意地址	输入
phy_addr	虚拟地址对应的物理地址，如果虚拟地址相对内存块由偏移，那么该物理地址也是相对偏移后的物理地址	输出

版权所有 © 上海云集互联有限公司

【返回值】

返回值	描述
0	成功。
非0	失败，其值为错误码。

【需求】

- 头文件：ot_smr_common.h、ot_smr.h
- 库文件：libot_smr.a

【注意】

- 虚拟地址可以是ot_smr_alloc申请的内存块中对应的任意用户态空间的地址，但是size不能超过申请的内存块地址范围。
- 不支持跨进程查询，即A进程申请的内存块不支持在B进程中查询

【举例】无。

【相关主题】

ot_smr_alloc

4.8. ot_smr_is_cached

【描述】

查询该段内存是否设置了缓存。

【语法】

```
ot_bool ot_smr_is_cached(ot_void* virt_addr);
```

参数名称	描述	输入/输出
virt_addr	虚拟地址，由ot_smr_mmap 映射生成的虚拟地址，虚拟地址可为内块空间内的任意地址	输入

版权所有 © 上海云集互联有限公司

【返回值】

返回值	描述
0	成功。
非0	失败，其值为错误码。

【需求】

- 头文件：ot_smr_common.h、ot_smr.h
- 库文件：libot_smr.a

【注意】

- 等同于ot_smr_virt2phy。

【举例】无。

【相关主题】

ot_smr_mmap　ot_smr_virt2phy

4.9. ot_smr_flush_cache

【描述】

刷新cache里的内容到内存并且使cache里的内容无效。

【语法】

```
ot_s32 ot_smr_flush_cache(ot_phys_addr phy_addr, ot_void* virt_addr, ot_u32 size);
```

【参数】

参数名称	描述	输入/输出
phy_addr	待操作内存的起始物理地址	输入
virt_addr	待操作数据的起始虚拟地址指针。 必须为用户态地址，且不能为空。 虚拟地址必须为物理地址映射的地址。	输入
size	待操作数据的大小。 不能为0。	

版权所有 © 上海云集互联有限公司

【返回值】

返回值	描述
0	成功。
非0	失败，其值为错误码。

【需求】

- 头文件：ot_smr_common.h、ot_smr.h
- 库文件：libot_smr.a

【注意】

- 当cache里的数据为最新数据时，为了保证不能直接访问cache的硬件（如cve）在访问内存时能够得到正确的数据，此时需要先调用接口将cache里的内容更新到内存，这样，当硬件访问内存时，保证了数据的一致性和正确性。
- 此接口应与ot_smr_mmap接口配套使用

【举例】无。

【相关主题】无

版权所有 © 上海云集互联有限公司

5. 数据类型

SMR相关数据类型定义如下：

- ot_smr_alloc_attr：定义smr的内存申请时的入参属性。

5.1. ot_smr_alloc_attr

【说明】

定义smr的内存申请时的入参属性。

【定义】

```
typedef struct {
    ot_s8 region_name[OT_SMR_REGION_NAME_LEN_MAX];
    ot_s8 chunk_name[OT_SMR_CHUNK_NAME_MAX];
    ot_u64 len;
    ot_u32 align;
    ot_bool cached;
} ot_smr_alloc_attr;
```

【成员】

成员名称	描述
region_name	通过name指定从哪个内存区域申请内存，当不确定或者没有强要求时，可以填空，从默认区域进行分配
chunk_name	通过name指定要申请的内存块的名字
len	需要申请的内存大小，单位字节
align	要申请的内存块对齐配置
cached	设定申请的内存是否需要缓存，OT_TRUE缓存，OT_FALSE则不缓存

【注意事项】无。

【相关数据类型及接口】

ot_smr_alloc

版权所有 © 上海云集互联有限公司

6. 错误码

提供的错误码如下表所示：

错误代码	宏定义	描述
0x8003000	OT_ERR_SMR_ILLEGAL_PAARAM	输入参数非法
0x8003002	OT_ERR_SMR_NULL_PTR	输入参数存在空指针
0x8003003	OT_ERR_SMR_NOT_SUPPORT	操作不支持
0x8003009	OT_ERR_SMR_NO_MEM	内存空间不足
0x8003020	OT_ERR_SMR_NOT_INIT	SMR还未进行初始化
0x8003021	OT_ERR_SMR_DEPEND_NOT_READY	SMR还未配置
0x8003022	OT_ERR_SMR_BUSY	SMR busy暂时无法处理

版权所有 © 上海云集互联有限公司